

You are helping me write a PLAN.md for one CCA-F exam scenario build. I will tell you which scenario, and give you access to the built code. Your job is to produce a single Markdown file that a senior reviewer could read and immediately know what the build teaches, where every idea lives in the code, and how to run it fail-first.

Follow the structure, tone, and rules below exactly. Do not invent sections. Do not skip sections. Do not soften the tone.

=== VOICE AND TONE RULES ===

- Plainspoken senior-engineer voice. Short declarative sentences.
- No marketing verbs (leverage, unlock, empower, seamless, powerful, robust).
- No emojis. No horizontal decoration beyond ---.
- Em dashes (--) for asides, used often.
- Every claim earns its place. If a sentence adds no fact, delete it.
- Contrast, do not decorate. Frame ideas as "X, not Y". Name the anti-pattern every time -- silent failure, lost provenance, super-agent, averaging conflicts, assumed inheritance, sequential-by-habit, narrow decomposition, speculative caching.
- Point at code, not concepts. Every architectural claim must cite file:line.
- Measured, not claimed. Every performance claim carries a number ("130s of work in 103s of waiting", "\$0.0941 / 94s"). No adjectives instead.
- Bold is for the noun being defined, not for shouting.
- If you do not know a line number, write "(line unknown -- verify)". Never invent one.

=== REQUIRED TOP MATTER (in this exact order) ===

Open the file with these labelled fields, each on its own line, bold label:

- ****Folder:**** folder name in backticks
- ****Exam:**** CCA-F, Scenario N (Name)
- ****Built on:**** the stack in one line
- ****Commands and expected output:**** `cmd.txt`
- ****Status:**** built / not built / verified against live run -- with a section pointer if corrections happened

Then a blank line, then:

- ****One line:**** italicised one-sentence description of what the build does end-to-end. Must name the input, the mechanism, and the output artifact.

Then a blank line, then:

- ****Sibling build:**** name the other cohort scenario folder in backticks. State its lesson in one italic sentence. Then on the next line, a blockquote stating THIS build's lesson in one sentence. The two lessons must contrast.

=== REQUIRED SECTIONS, IN THIS ORDER (do not renumber, do not merge) ===

1. The problem it actually solves

Open with a realistic user question in italics. Then a bulleted list of THE FOUR failure modes a naive version produces. Close with one line: "each fix is a domain the exam tests."

2. Domain weightage for this scenario

Table with columns: Domain | Weight | Why this scenario loads it.

Rows in weight order, largest first. Weights sum to 100%. Bold the top two.

End with one line stating the combined weight of the top two

(e.g. "D1 + D5 = 70% of this scenario").

3. What is covered, and where it lives

One subsection per domain in weight order. Each subsection is a table:

Concept | Where | Anti-pattern you can run.

"Where" is `file: symbol` or `file: line`. "Anti-pattern" is the CLI flag or the concrete wrong thing. For domains at 10% weight or below, two or three sentences are acceptable in place of the table, and MUST say the domain is "thin on purpose" and which sibling scenario carries it.

4. Architecture

An ASCII diagram fenced with triple backticks, showing:

- the hub
- each worker with its tool grant printed on the diagram itself
- what flows back to the hub
- the final artifact path

Close with ONE sentence naming the architectural constraint that makes every other section make sense (e.g. "Each box is its own session with its own tool grant and its own prompt -- they share nothing").

5. What the hub does, step by step

Numbered table: Step | What happens | Where. Six rows: READ, DECOMPOSE, DISPATCH, COLLECT, CHECK, REPORT (or the six-step equivalent for the scenario).

End with ONE sentence naming which steps are code vs which are prompts, and why.

6. The N workers / agents / components

Table: Component | Tools | Does | Must not.

The "Must not" column is REQUIRED -- it is where anti-patterns get preempted.

Refuse to omit it. Follow with the required output shape as a fenced code block if the scenario writes an artifact.

7. The data contract

A fenced code block showing the data classes with fields (comment fields inline).

Follow with one sentence naming the ONE distinction the exam asks about (e.g. "access_failure vs empty_result").

If there is an input file format, show it in a second fenced block.

End with a "Key functions" table: Function | Does. One row per function that enforces an invariant. Emphasize functions that RAISE rather than warn.

8. Files

FULL PROJECT FOLDER STRUCTURE, mandatory.

Render as a fenced code block containing the actual repo tree. Walk the real repository -- do NOT infer files from context. For every file, add an inline right-aligned comment describing what it holds in fewer than 10 words.

Must include:

- all top-level files (pyproject.toml, .env.example, cmd.txt, PLAN.md, README if present)
- the agents/ (or workers/) subdirectory and every agent file with its role
- sample_queries/ or equivalent input directory (list files, do not expand contents)
- documents/ or equivalent (with a note on what file types it holds)
- output/ (with a note on the stable path and any timestamped per-run directories)
- tests/ (with test count if known)

Exclude .venv, __pycache__, .git, .DS_Store, and any editor metadata.

If a file exists in the repo but you do not know its purpose, list it with the comment "(purpose unknown -- verify)". Do NOT invent a description.

9. How to run it

Open with ONE sentence naming the directory-context gotcha (e.g. "run everything from inside this folder"). Then the setup block (uv venv, uv pip install, .env).

Then a table: Command | Key? | Does. Then a "Flags" line, "Exit code" line, "Output" line. Then per-agent caps (MAX_TURNS, BUDGET_USD, TIMEOUT_S) if any.

If cost was measured before and after optimization, include a before/after table with real numbers and one paragraph naming what the change was.

End with a UI description if there is one.

10. Sample inputs and the use case behind each

Table: Query file (or input) | Use case | What would go wrong without this.

Every input file the build ships with must appear. Close with one line stating that the input list is the yardstick the run is graded against.

11. How to see each idea fail

Prose intro: "Run once correctly, then flip ONE switch. Two at once tells you nothing." Then a table: Switch | CLI | What you see.
Then a numbered list of the offline checks the demo mode shows.
Then a tests section: pytest results and one table File | Tests | Covers.

12. Not built

Bulleted list of things deliberately not built and why. Name each. Do not apologise for them.

13. Anti-patterns the exam offers as plausible answers

Numbered list. Each entry is a short phrase in bold followed by an em-dashed concrete example. 6-10 entries. No generic entries.

14. Status

2-4 sentences. State what runs offline (no API key), what runs live (needs key), and what has been verified against a real run.

15. Demo map -- every concept, and the exact line that shows it

The centrepiece. Open with: "Open the file, jump to the line, point at it.

Line numbers are from this commit; if you edit the code, re-check with grep -n."

One subsection per domain in weight order. Each subsection is a table:

| Concept | Where | What to say standing at that line.

Every row's "Where" must be file:line. Every row's "What to say" must be a sentence a teacher would actually say standing in front of that line -- not a description of the line. Bold the 6-8 highest-signal concept names across the whole demo map.

Close with a small table: Switch | CLI | Line that changes behaviour -- for each anti-pattern flag from section 11.

16. What the live run changed

The single most valuable section of the plan. Never skip.

Write it as a story, not a changelog. Open by describing what the textbook answer was, then say what happened when it was run for real. Enumerate each correction (there are usually 3-6). For each correction:

- name what the textbook said
- name what the live run showed
- name the fix
- name the concept that got stronger as a result

If the build had a signature failure -- a run that broke in exactly the way the build exists to prevent -- give it its own paragraph. That is the highest-value teaching moment in the entire plan.

Close with one sentence stating the lesson plainly, in one line.

If the build genuinely had no live-run corrections, this section must still exist and must say so explicitly, plus what was verified end-to-end and how.

=== REFUSAL CRITERIA ===

Refuse to:

- use marketing adjectives anywhere
- invent file names or line numbers you cannot verify from the code I gave you
- omit the "Must not" column from the components table (section 6)
- render section 8 as anything other than a fenced tree of real files
- collapse two sections into one
- write section 16 as a bullet list of "improvements"
- skip any of the 16 numbered sections

=== YOUR TURN ===

Now write the PLAN.md for the scenario I am about to name, using the code I am about to give you access to. Do not summarise the code back to me. Do not ask clarifying questions unless a required section literally cannot be filled from the code. Produce the PLAN.md, and only the PLAN.md.